

PharmaDT — Detailed Implementation Guide

Step-by-Step Build Flow with Cryptographic Provenance Ledger

Project: PharmaDT — An Agentic AI Framework Integrating Digital Twin Simulation, Cryptographic Provenance, and Federated Learning for Resilient Drug Supply Chains **Team:** S Sanjana (1BM23AI159) · Sandeep N (1BM23AI169) · Sai Shreekar G (1BM23AI164) **Guide:** Dr. Monika Puttaramaiah, Associate Professor, Dept. of Machine Learning, BMSCE **Scope of this document:** Stage-by-stage execution plan derived from your High-Level Design and Low-Level Design diagrams, with Hyperledger Fabric replaced by a Cryptographic Provenance Ledger.

PART A — What Changes in Your Existing Diagrams

Hyperledger Fabric is removed. Every property it provided is preserved by the **Cryptographic Provenance Ledger (CPL)** — a permissioned, append-only, hash-chained, ECDSA-signed ledger implemented in Python over PostgreSQL. Nothing else in your architecture changes. The table below is your edit list; apply these to the diagram sources before Review 3.

#	Diagram	Existing element	Replace with	Notes
A1	Overall Architecture (6-layer)	Layer 4 box: "Hyperledger Blockchain — Tamper-proof audit trail"	"Cryptographic Provenance Ledger — Hash-chained + ECDSA-signed audit trail"	Layer name "Trust + Learning Layer" stays. Layer count stays 6.
A2	HLD	Panel titled "BLOCKCHAIN LAYER"	"PROVENANCE LAYER"	Panel colour/position unchanged
A3	HLD	Node "Hyperledger Fabric"	"PostgreSQL Append-Only Store + ECDSA Keyring"	This is the only <i>implementation</i> node that changes
A4	HLD	Node "Immutable Transaction Log"	"Hash-Chained Record Log"	Property retained, mechanism renamed
A5	HLD	Nodes "Anti-Counterfeit Verification", "Batch Tracking", "Drug Provenance"	Unchanged	These are functional capabilities, still delivered
A6	HLD	Edge label "verified data"	Unchanged	Feedback loop into agents is preserved
A7	LLD — Agent 5 (Anomaly Detection)	"Blockchain Hash Verify — ledger cross-check"	"Ledger Hash Verify — SHA-256 chain cross-check"	Same position in flow, same "mismatch → Counterfeit / Fraud Alert" output
A8	AI Agents Architecture	"Counterfeit Flags" edge from Anomaly Agent	Unchanged	Source of truth is now CPL instead of Fabric
A9	Report §5.2.4	Section heading "Blockchain Provenance Protocol"	"Cryptographic Provenance Protocol"	Also fixes the copy-paste error in your report — §5.2.4 currently repeats the MADDPG text from §5.2.2 and must be rewritten regardless
A10	Report Software Requirements	"Hyperledger Fabric 2.5, Hyperledger Explorer, Docker peer/orderer nodes"	"cryptography 42.x (ECDSA P-256), hashlib, PostgreSQL 15 append-only schema"	Removes ~6 heavy dependencies
A11	Objective 3	"Integrate blockchain-based provenance tracking using Hyperledger Fabric"	"Integrate cryptographic provenance tracking using a permissioned hash-chained ledger"	Objective count stays 6

The one-paragraph justification (use this verbatim in your report and viva)

*In a simulated single-consortium supply chain there are no mutually distrusting, independently operated validator organisations. Distributed Byzantine consensus is therefore over-provisioned for the threat model this project actually addresses. PharmaDT implements a permissioned, append-only Cryptographic Provenance Ledger providing tamper-evidence via SHA-256 hash chaining, non-repudiation and write-authorisation via per-node ECDSA (NIST P-256) digital signatures, and efficient inclusion proofs via Merkle-root anchoring. This satisfies FR-07, NFR-04 and NFR-08 in full while eliminating operational overhead orthogonal to the research contribution. The ledger is accessed exclusively through an abstract **ProvenanceLedger** interface, making the system forward-compatible with a Hyperledger Fabric backend without modification to the agent layer.*

Property-preservation proof (put this table in your report)

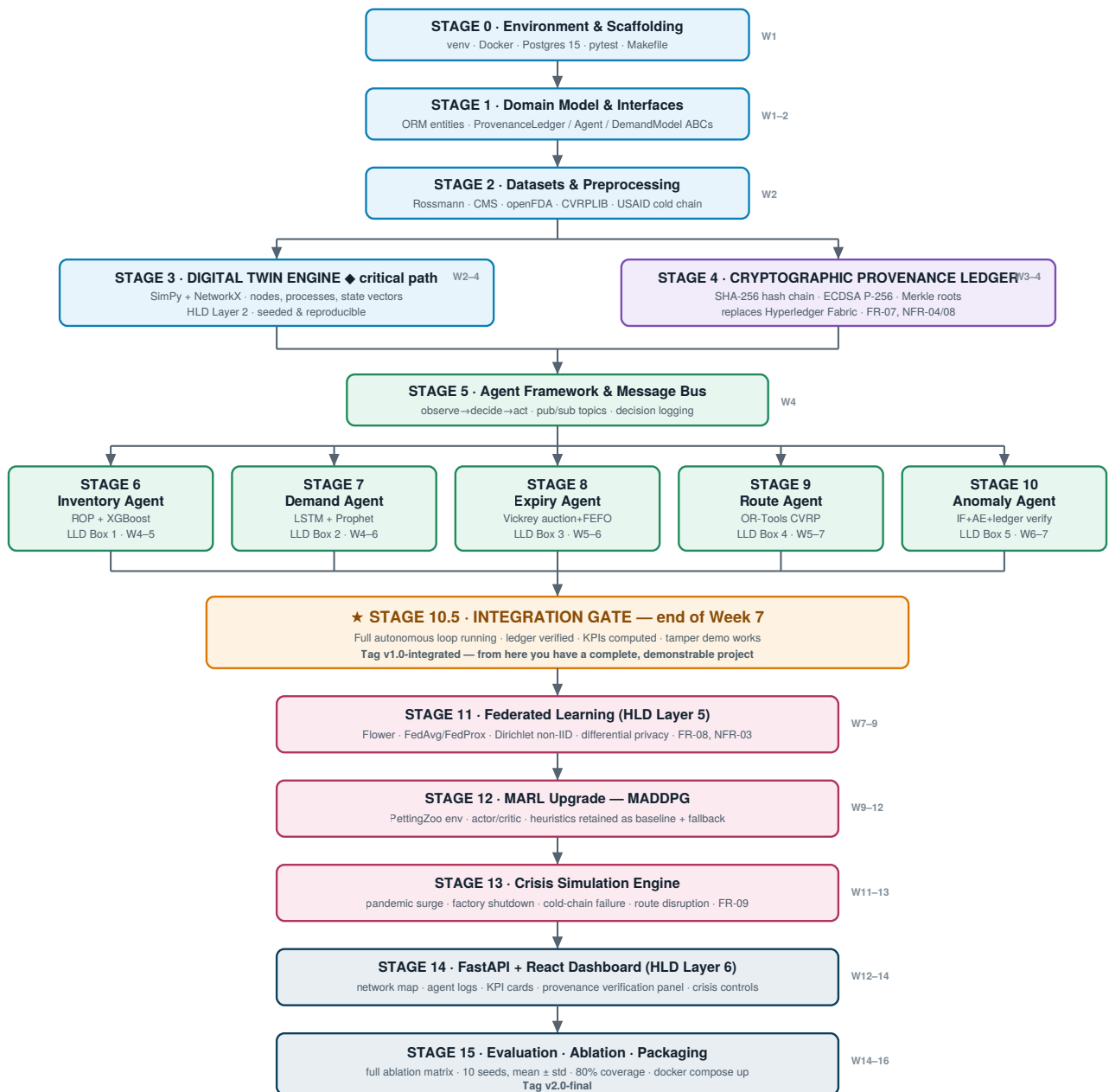
Requirement	Fabric mechanism	CPL mechanism	Preserved?
FR-07 immutable batch + handoff record	Block + world state	Hash-chained append-only row	Yes
NFR-04 cryptographically signed txn	MSP + X.509 CA	ECDSA P-256 per-node keypair	Yes
NFR-04 only authorised peers write	Channel policy	Public-key allow-list in registry	Yes
NFR-08 immutable audit trail	Ledger history	<code>verify_chain()</code> + DB trigger blocking UPDATE/DELETE	Yes
Anti-counterfeit hash check	Chaincode	<code>verify_batch_hash()</code> SHA-256 fingerprint	Yes
Gap 3.2.3 ledger events feed agents	Chaincode events	Direct ledger event subscription	Yes
Byzantine fault tolerance $f=(n-1)/3$	Raft/BFT ordering	Dropped — out of threat model	No (intentional)

PART B — Master Build Flowchart

The flowchart on the next page is the authoritative order of work. Read it as: everything above a box must be complete before that box starts. Two tracks run in parallel (Stage 3 and Stage 4) and five run in parallel (Stages 6–10).

The single most important feature of this flow is the ★ INTEGRATION GATE at Stage 10.5. At that point you have a complete, autonomous, demonstrable system that satisfies the majority of your functional requirements. Everything after the gate adds research depth. If time runs out after the gate, you still have a complete project. If you sequence the work any other way, you risk having many half-finished parts and nothing that runs.

Master Build Flowchart



TRACK LEGEND

Foundation	Provenance ledger (replaces blockchain)	Agents (HLD Layer 3 / LLD boxes 1-5)	Milestone gate
Research depth (optional — cut from the end if time runs short)		Delivery & evaluation	

PART C — Repository Layout

Create this exact structure in Stage 0. Every later stage tells you which file it fills in.

```

pharmadt/
├─ docker-compose.yml      # postgres + api + dashboard
├─ Dockerfile
├─ requirements.txt
├─ pyproject.toml
├─ .env.example
├─ README.md
├─ Makefile                # make sim / make test / make train
├─
├─ pharmadt/
│   ├── __init__.py
│   ├── config.py          # pydantic-settings; all constants live here
│   ├──
│   ├── core/              # — STAGE 1 —
│   │   ├── models.py      # SQLAlchemy ORM: Drug, Batch, Node, Shipment...
│   │   ├── schemas.py     # Pydantic DTOs
│   │   ├── interfaces.py  # ABCs: ProvenanceLedger, Agent, DemandModel
│   │   ├── events.py      # Event dataclasses + EventType enum
│   │   └─ db.py           # engine, session factory
│   ├──
│   ├── twin/              # — STAGE 3 — Layer 2 of HLD
│   │   ├── network.py     # NetworkX graph G=(V,E)
│   │   ├── nodes.py       # Manufacturer/Warehouse/Distributor/Pharmacy
│   │   ├── simulation.py  # SimPy Environment + main loop
│   │   ├── processes.py   # demand, shipment, expiry, cold-chain processes
│   │   └─ state.py        # state-vector extraction for agents
│   ├──
│   ├── ledger/            # — STAGE 4 — replaces Fabric
│   │   ├── crypto.py      # SHA-256 canonicalisation, ECDSA sign/verify
│   │   ├── keyring.py     # per-node keypair generation + registry
│   │   ├── chain.py       # ProvenanceLedger implementation
│   │   ├── merkle.py      # Merkle tree + inclusion proofs
│   │   └─ schema.sql      # append-only table + immutability triggers
│   ├──
│   ├── agents/            # — STAGES 5-10 — Layer 3 of HLD
│   │   ├── base.py        # Agent ABC: observe() → decide() → act()
│   │   ├── bus.py         # publish/subscribe message bus
│   │   ├── inventory.py   # LLD box 1
│   │   ├── demand.py      # LLD box 2
│   │   ├── expiry.py      # LLD box 3
│   │   ├── routing.py     # LLD box 4
│   │   └─ anomaly.py      # LLD box 5
│   ├──
│   ├── ml/
│   │   ├── lstm.py        # PyTorch demand LSTM
│   │   ├── prophet_model.py # Prophet wrapper
│   │   ├── regression.py  # XGBoost dynamic reorder threshold
│   │   ├── isolation_forest.py
│   │   ├── autoencoder.py
│   │   └─ preprocessing.py
│   ├──
│   ├── federated/         # — STAGE 11 — Layer 5 of HLD
│   │   ├── client.py      # flwr NumPyClient
│   │   ├── server.py     # FedAvg / FedProx strategy
│   │   ├── partition.py   # non-IID Dirichlet split
│   │   └─ privacy.py      # Gaussian DP noise + budget accounting
│   ├──
│   ├── marl/              # — STAGE 12 —
│   │   ├── env.py         # PettingZoo ParallelEnv wrapper
│   │   ├── maddpg.py      # actor/critic networks + training loop
│   │   ├── spaces.py      # observation/action space definitions
│   │   └─ rewards.py      # reward functions per report §5.2.2
│   ├──
│   ├── crisis/            # — STAGE 13 —
│   │   ├── scenarios.py   # YAML-driven scenario definitions
│   │   └─ injector.py     # external event injection into SimPy
│   ├──
│   └─ api/                # — STAGE 14 — Layer 6 of HLD
│       └─ main.py          # FastAPI app

```

```

|   |   └─ routes/           # /simulation /agents /ledger /kpi /crisis
|   |   └─ websocket.py      # live event stream to dashboard
|
| └─ dashboard/              # React + Vite
|   └─ src/
|     └─ components/        # NetworkMap, AgentLog, KPICards,
|     |                     # ProvenanceTrace, CrisisControls
|     └─ hooks/useSimulationSocket.js
|
| └─ data/
|   └─ raw/                  # downloaded datasets (gitignored)
|   └─ processed/
|   └─ scenarios/*.yaml
|
| └─ notebooks/              # EDA + model prototyping only
| └─ experiments/            # configs + results + ablations
└─ tests/                    # pytest; target 80% coverage (NFR-07)

```

Why this structure matters for your evaluation: NFR-07 requires independently deployable modules and 80% test coverage. This layout makes each layer of your HLD a separate importable package with no circular dependencies, which is exactly what an examiner looks for when judging whether the architecture diagram matches the code.

PART D — Stage-by-Stage Execution

Each stage gives you: **Goal**, **Where** (files), **Tools**, **Exact steps**, **Definition of Done (DoD)**, and **Pitfalls**. Do not start a stage until the previous DoD passes.

STAGE 0 — Environment & Scaffolding

Week 1 · Owner: all three (pair once, then split)

Goal

A repository that boots, connects to a database, and runs a passing test — before any domain logic exists.

Tools

Python 3.11 (not 3.12 — Prophet and some RL libraries lag), Git, Docker Desktop, PostgreSQL 15 (via Docker), VS Code + Python extension, `uv` or `venv`.

Exact steps

1. **Create repo and environment.** `bash mkdir pharmadt && cd pharmadt && git init python3.11 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate pip install --upgrade pip`
2. **Pin dependencies.** Create `requirements.txt`:
`simpy==4.1.1 networkx==3.3 sqlalchemy==2.0.31 psycpg2-binary==2.9.9 alembic==1.13.2 pydantic==2.8.2 pydantic-settings==2.4.0 fastapi==0.112.0 uvicorn[standard]==0.30.5 cryptography==43.0.0 pandas==2.2.2 numpy==1.26.4 scikit-learn==1.5.1 xgboost==2.1.0 torch==2.4.0 prophet==1.1.5 ortools==9.10.4067 flwr==1.10.0 pettingzoo==1.24.3 gymnasium==0.29.1 pyyaml==6.0.2 pytest==8.3.2 pytest-cov==5.0.0` Then `pip install -r requirements.txt`.
3. **Start PostgreSQL.** `docker-compose.yml`: `yaml services: db: image: postgres:15 environment: POSTGRES_USER: pharmadt POSTGRES_PASSWORD: pharmadt POSTGRES_DB: pharmadt ports: ["5432:5432"] volumes: ["pgdata:/var/lib/postgresql/data"] volumes: { pgdata: }` Run `docker compose up -d db`.
4. **Config module** — `pharmadt/config.py` using `pydantic-settings`, reading `DATABASE_URL`, `SIM_SEED`, `SIM_DAYS`, `NUM_NODES`. **Never hardcode a constant anywhere else in the codebase.** Examiners notice magic numbers.
5. **Makefile** with `make test`, `make sim`, `make api`, `make lint`.
6. **Initialise pytest** with a trivial `tests/test_smoke.py` asserting the DB connects.

DoD

`make test` passes; `docker compose up -d db` succeeds; repo pushed to GitHub with a `.gitignore` covering `.venv/`, `data/raw/`, `*.pt`, `__pycache__/`.

Pitfalls

Do not install `prophet` before `numpy` — it compiles against the installed NumPy ABI. If Prophet fails on Windows, use WSL2; this is the single most common environment failure in projects like this.

STAGE 1 — Domain Model & Abstract Interfaces

Week 1–2 · Owner: A

Goal

Freeze the data contracts and the three abstract base classes. Everything downstream depends on these, so changing them later is expensive — spend real time here.

Where

`pharmadt/core/models.py`, `schemas.py`, `interfaces.py`, `events.py`, `db.py`

Entities (SQLAlchemy ORM)

Entity	Key fields
Drug	drug_id, name, atc_code, shelf_life_days, requires_cold_chain, temp_min_c, temp_max_c, is_controlled
Node	node_id, node_type (MANUFACTURER/WAREHOUSE/DISTRIBUTOR/PHARMACY/HOSPITAL), lat, lon, storage_capacity, has_cold_storage, public_key
Batch	batch_id, drug_id, manufacturer_id, mfg_date, expiry_date, quantity, batch_fingerprint (SHA-256)
InventoryRecord	node_id, batch_id, quantity_on_hand, sim_day
Shipment	shipment_id, from_node, to_node, batch_id, quantity, dispatch_day, eta_day, status, temp_log (JSONB)
DemandRecord	node_id, drug_id, sim_day, quantity_demanded, quantity_fulfilled
AgentDecision	decision_id, agent_name, sim_day, inputs (JSONB), action (JSONB), justification — this table alone satisfies NFR-08's agent-decision logging
ProvenanceRecord	filled in Stage 4

The three interfaces (write these first)

```
# pharmaDT/core/interfaces.py
from abc import ABC, abstractmethod

class ProvenanceLedger(ABC):
    @abstractmethod
    def record_event(self, batch_id, event_type, from_node,
                    to_node, payload, signer_node) -> str: ...
    @abstractmethod
    def get_provenance(self, batch_id) -> list: ...
    @abstractmethod
    def verify_chain(self, start=None, end=None) -> bool: ...
    @abstractmethod
    def verify_batch_fingerprint(self, batch_id, presented) -> bool: ...

class Agent(ABC):
    name: str
    @abstractmethod
    def observe(self, world_state) -> dict: ...
    @abstractmethod
    def decide(self, observation) -> list: ... # list[Action]
    @abstractmethod
    def act(self, actions, world) -> None: ...

class DemandModel(ABC):
    @abstractmethod
    def fit(self, X, y) -> None: ...
    @abstractmethod
    def predict(self, X, horizon: int = 14): ...
    @abstractmethod
    def get_weights(self): ... # required by Flower in Stage 11
    @abstractmethod
    def set_weights(self, w) -> None: ...
```

Why `get_weights` / `set_weights` now: federated learning in Stage 11 only works if every demand model exposes weight serialisation. Adding it retroactively means rewriting the model classes.

DoD

`alembic upgrade head` creates all tables; a seed script inserts 12 nodes, 5 drugs, 20 batches; `pytest tests/test_models.py` passes.

STAGE 2 — Dataset Acquisition & Preprocessing

Week 2 · Owner: C (runs parallel to Stage 3)

Datasets, sources, and exact use

Dataset	Where to get it	Used by	Preprocessing required
Rossmann Store Sales (1.1M rows, 1115 stores)	Kaggle: c/rossmann-store-sales	Demand Agent LSTM + Prophet	Parse dates; filter <code>Open==1</code> ; per-store series; log1p transform; train/val/test split by time (never random — that leaks)
CMS Medicare Part D Utilisation	data.cms.gov	FL validation, drug-level demand	Aggregate to drug × state × year; map to your 5 synthetic drugs
openFDA Drug Recall	api.fda.gov/drug/enforcement.json	Anomaly Agent labels	Extract <code>reason_for_recall</code> , <code>classification</code> ; derive binary anomaly labels
CVRPLIB	vrp.atd-lab.inf.puc-rio.br	Route Agent benchmark	Standard <code>.vrp</code> parser; validate OR-Tools solution quality vs known optima
USAID cold chain temperature	USAID Deliver / VillageReach open data	Cold-chain excursion simulation	Extract excursion event distributions to parameterise the twin
Kaggle Supply Chain	Kaggle: supply-chain analysis dataset	Inventory simulation priors	Lead times, defect rates as distribution parameters

Steps

1. `data/raw/` for downloads (gitignored — never commit datasets), `data/processed/` for Parquet outputs.
2. Write `pharmadt/ml/preprocessing.py` with one function per dataset returning a tidy DataFrame. **No notebook logic in the pipeline** — notebooks are for exploration only, functions are for the system.
3. Save processed outputs as Parquet (`df.to_parquet`) — 10× faster reload than CSV during model iteration.
4. Fit a demand distribution per (node, drug) from Rossmann to drive the twin's stochastic demand in Stage 3.

DoD

`python -m pharmadt.ml.preprocessing --all` produces every Parquet file; an EDA notebook documents seasonality, weekday effects, and missingness.

Pitfall

Rossmann is *retail store* sales, not pharmaceutical demand. State this limitation explicitly in your report as a validity threat and mitigate it by validating demand patterns against CMS drug-level data. An examiner will ask this.

STAGE 3 — Digital Twin Engine ← Layer 2 of your HLD

Weeks 2–4 · Owner: A · CRITICAL PATH

Goal

A SimPy discrete-event simulation that runs the supply-chain network forward in time and emits an event stream. Everything else consumes this.

Where

`pharmadt/twin/network.py` , `nodes.py` , `simulation.py` , `processes.py` , `state.py`

Steps

1. **Network graph** (`network.py`). Build `G = nx.DiGraph()`. Node attributes: type, capacity, cold storage, lat/lon. Edge attributes: `distance_km` , `transit_days` , `cost_per_unit` , `cold_chain_capable`. Start with **12 nodes** (1 manufacturer, 2 warehouses, 3 distributors, 6 pharmacies). Scale to 50 only after everything works — FR-01 requires 50, but debugging at 50 is painful.
2. **Node state vector** (`state.py`). Per report §5.2.1, each node exposes: current stock per drug, pending inbound orders, temperature zone, storage utilisation, rolling demand history (last 28 days). This vector is *literally* the agent observation space in Stage 12 — design it once, use it twice.
3. **SimPy processes** (`processes.py`), one generator function each:
4. `demand_process(env, node)` — samples daily demand from the Stage-2 fitted distribution; decrements stock; records fulfilled vs unfulfilled (**stockout KPI**).
5. `order_process(env, node)` — consumes replenishment orders published by the Inventory Agent.
6. `shipment_process(env, shipment)` — `yield env.timeout(transit_days)` ; on arrival, increments stock and **writes a ledger handoff record**.
7. `expiry_process(env, node)` — daily scan; removes expired stock (**wastage KPI**).
8. `coldchain_process(env, shipment)` — samples temperature each step; emits excursion events.
9. **Main loop** (`simulation.py`). Time step = 1 simulated day (configurable, per §5.2.1). `env.run(until=SIM_DAYS)`. Emit every event to an in-memory event log **and** the message bus.
10. **Reproducibility**. Seed `random` and `numpy.random` from `config.SIM_SEED`. Two runs with the same seed must produce byte-identical event logs. Assert this in a test.

DoD

`make sim` runs 365 simulated days over 12 nodes in under 30 seconds, emits a CSV/JSON event log, and prints baseline KPIs (stockout rate, wastage units, average inventory). Same seed → identical output.

Pitfalls

- SimPy generators must `yield`; a missing `yield` produces a silently non-running process. Test each process in isolation.
- Do not couple agents to SimPy internals. Agents receive a plain `world_state` dict from `state.py`. This decoupling is what makes the MARL wrapper in Stage 12 possible.
- NFR-01 demands 1000 time steps/second. Do not write to PostgreSQL inside the simulation loop — buffer events in memory and bulk-insert at the end.

STAGE 4 — Cryptographic Provenance Ledger ← *replaces the Blockchain Layer*

Weeks 3–4 · Owner: C · runs parallel to Stage 3

Goal

A permissioned, append-only, tamper-evident, digitally signed provenance ledger delivering FR-07, NFR-04, NFR-08 and the anti-counterfeit check.

Where

`pharmadt/ledger/crypto.py`, `keyring.py`, `chain.py`, `merkle.py`, `schema.sql`

4.1 — Database schema (append-only, enforced by the database itself)

```
CREATE TABLE provenance_records (
    seq          BIGSERIAL PRIMARY KEY,
    batch_id     TEXT NOT NULL,
    event_type   TEXT NOT NULL,
    from_node    TEXT,
    to_node      TEXT,
    payload      JSONB NOT NULL,
    sim_day      INTEGER NOT NULL,
    recorded_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    prev_hash    CHAR(64) NOT NULL,
    record_hash  CHAR(64) NOT NULL UNIQUE,
    signer_node  TEXT NOT NULL,
    signature    TEXT NOT NULL,
    merkle_root  CHAR(64)
);
CREATE INDEX idx_prov_batch ON provenance_records(batch_id);

-- Immutability enforced at the database layer, not in application code
CREATE OR REPLACE FUNCTION block_mutation() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'provenance_records is append-only'; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER no_update BEFORE UPDATE ON provenance_records
    FOR EACH ROW EXECUTE FUNCTION block_mutation();
CREATE TRIGGER no_delete BEFORE DELETE ON provenance_records
    FOR EACH ROW EXECUTE FUNCTION block_mutation();
```

Say this in your viva: immutability is enforced by a database trigger, so it holds even against the application layer — not merely by convention.

4.2 — Canonical serialisation and hashing (`crypto.py`)

```
import json, hashlib

def canonical(obj: dict) -> bytes:
    # Deterministic: identical dict -> identical bytes, always
    return json.dumps(obj, sort_keys=True, separators=(",", ":"),
        default=str).encode()

def sha256_hex(b: bytes) -> str:
    return hashlib.sha256(b).hexdigest()

def compute_record_hash(payload_fields: dict, prev_hash: str) -> str:
    return sha256_hex(canonical(payload_fields) + prev_hash.encode())
```

`sort_keys=True` is non-negotiable — without it, dict ordering changes the hash and your chain verification fails intermittently.

4.3 — ECDSA keyring (`keyring.py`)

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes, serialization

def generate_node_keypair():
    priv = ec.generate_private_key(ec.SECP256R1()) # NIST P-256
    pub_pem = priv.public_key().public_bytes(
        serialization.Encoding.PEM,
        serialization.PublicFormat.SubjectPublicKeyInfo).decode()
    return priv, pub_pem

def sign(priv, record_hash: str) -> str:
    return priv.sign(record_hash.encode(),
        ec.ECDSA(hashes.SHA256())).hex()

def verify(pub, record_hash: str, sig_hex: str) -> bool:
    from cryptography.exceptions import InvalidSignature
    try:
        pub.verify(bytes.fromhex(sig_hex), record_hash.encode(),
            ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False
```

Each `Node` row stores its public key (added in Stage 1). The registry of public keys **is** your permissioning layer: a record signed by a key not in the registry is rejected. This is the direct replacement for Fabric's MSP + channel policy.

4.4 — The ledger (`chain.py`)

Implement `ProvenanceLedger` from Stage 1: - `record_event(...)`: fetch `prev_hash` (the `record_hash` of the highest `seq`, or 64 zeros for genesis) → build the field dict → `compute_record_hash` → sign with the signer node's private key → INSERT. - `verify_chain()`: walk records in `seq` order; recompute each hash; check it equals the stored `record_hash`; check `prev_hash` links correctly; verify each signature against the signer's registered public key. Return `False` at the first break, with the offending `seq`. - `get_provenance(batch_id)`: all records for a batch in `seq` order — the manufacturer-to-patient trace. - `verify_batch_fingerprint(batch_id, presented)`: recompute `SHA256(batch_id || drug_id || manufacturer_id || mfg_date || expiry_date || quantity)` and compare. Mismatch ⇒ counterfeit flag (this feeds LLD Agent 5).

Event types to record: `BATCH_CREATED`, `SHIPMENT_DISPATCHED`, `SHIPMENT_RECEIVED`, `COLD_CHAIN_EXCURSION`, `REDISTRIBUTION`, `DISPENSED`, `RECALLED`.

4.5 — Merkle anchoring (`merkle.py`)

Every `N = 64` records, build a Merkle tree over those record hashes, store the root on the 64th record. Provide `inclusion_proof(seq)` returning the $O(\log n)$ sibling path and `verify_inclusion(record_hash, proof, root)`. This is optional for functionality but **strongly recommended** — it is the element that makes the ledger read as cryptographically serious rather than as "just a hash column".

4.6 — Wire into the twin

In `shipment_process` and batch creation, call `ledger.record_event(...)`. Every physical handoff in the simulation now has a signed, chained ledger entry.

DoD (demo this in your review — it is your most persuasive slide)

1. Run the simulation; ledger accumulates thousands of records.
2. `verify_chain()` → `True`.
3. Tamper: `ALTER TABLE provenance_records DISABLE TRIGGER no_update;` then modify one `payload` field.
4. `verify_chain()` → `False`, reporting the exact `seq` where the chain breaks.
5. `get_provenance(batch_id)` prints the full manufacturer→pharmacy trace.
6. `verify_batch_fingerprint` with a forged hash → counterfeit detected.

Pitfalls

- Genesis record: define `prev_hash = "0"*64` explicitly.
- Concurrency: if two processes append simultaneously the chain forks. Serialise appends with a `SELECT ... FOR UPDATE` on a lock row, or a single-threaded ledger writer. Your simulation is single-threaded, so this is simple — but *mention* it, because an examiner may probe it.
- Store the private keys in `data/keys/` (gitignored). Note honestly in your report that in production these belong in an HSM or KMS.

STAGE 5 — Agent Framework & Message Bus

Week 4 · Owner: B

Goal

The skeleton that all five agents plug into — matching the "collaborate" edges in your AI-Agents-Architecture diagram.

Where

`pharmadt/agents/base.py`, `bus.py`

Steps

1. **BaseAgent** implements the **Agent** ABC with the canonical cycle: `observe(world_state) → decide(obs) → act(actions, world)`. Every agent runs this once per simulated day.
2. **Message bus** (`bus.py`) — an in-process publish/subscribe registry: `subscribe(topic, handler)`, `publish(topic, message)`. Topics correspond exactly to the labelled edges in your agent diagram:

Topic	Publisher	Subscriber
<code>forecast.data</code>	Demand Agent	Inventory Agent
<code>shortage.alert</code>	Demand Agent	Inventory Agent
<code>demand.hotspot</code>	Demand Agent	Expiry Agent
<code>counterfeit.flag</code>	Anomaly Agent	Expiry Agent, Dashboard
<code>replenishment.order</code>	Inventory Agent	Route Agent
<code>redistribution.request</code>	Expiry Agent	Route Agent
<code>route.plan</code>	Route Agent	Digital Twin
<code>ledger.event</code>	Ledger	Anomaly Agent

Use Redis pub/sub only if you later split into separate processes; in-process is sufficient and far easier to debug. 3. **Decision logging**. `BaseAgent.act()` writes an `AgentDecision` row every time — inputs, action, justification, timestamp. **This single mechanism satisfies NFR-08's agent-decision auditability requirement**, so build it into the base class rather than into each agent. 4. **Orchestrator**. A `run_agents(world_state, day)` function invoked by the SimPy loop each day, executing agents in dependency order: Demand → Inventory → Expiry → Route → Anomaly.

DoD

A dummy agent subscribes, receives a published message, and writes an `AgentDecision` row. Bus unit tests pass.

STAGE 6 — Inventory Agent ← LLD Box 1

Weeks 4–5 · Owner: B

LLD flow being implemented

`Inventory Data Stream` + `Warehouse DB Interface` → `Stock Monitoring Engine` (continuous polling) → `Threshold Trigger` (stock < min?) → if low → `ML Regression Model` (Linear/XGBoost) → `REST API /replenish-order` → **Replenishment Order**

Steps

1. **Stock Monitoring Engine** — read `quantity_on_hand` per (node, drug) from the twin's world state each day.
2. **Threshold Trigger** — start with the classical reorder point: $ROP = (\text{mean_daily_demand} \times \text{lead_time_days}) + \text{safety_stock}$, where $\text{safety_stock} = z \times \sigma_{\text{demand}} \times \sqrt{\text{lead_time}}$ and $z = 1.65$ for 95% service level. Implement this first; it is your baseline.
3. **ML Regression Model** (`ml/regression.py`) — XGBoost regressor predicting the *optimal reorder quantity*. Features: 7/14/28-day rolling demand mean and std, day-of-week, days-to-expiry of current stock, forecast from the Demand Agent, lead time, current stock. Target: the quantity that would have minimised (stockout cost + holding cost) in hindsight — generate this label by running the simulation with a grid of quantities and selecting the argmin. Explain this label-generation procedure in your report; it is a genuine methodological contribution.
4. **REST endpoint** `/replenish-order` in Stage 14; internally publish to `replenishment.order`.

DoD

Stockout rate drops measurably versus a fixed-threshold policy. Log the comparison — it becomes an ablation row in Stage 15.

STAGE 7 — Demand Prediction Agent ← LLD Box 2

Weeks 4–6 · Owner: C

LLD flow

Local Hospital Data + Rossmann Sales / CMS → LSTM Network || Prophet Forecasting → (local weights) → Flower Federated Aggregator → global model → **14-Day Rolling Forecast**

Build the models **centrally first** (this stage), then federate them (Stage 11). Attempting both at once is the most common way this component stalls.

Steps

1. **LSTM** (`ml/lstm.py` , PyTorch). Input: 28-day lookback window of (demand, day-of-week, is_holiday, rolling mean). Architecture: 2 LSTM layers, hidden 64, dropout 0.2, linear head → 14 outputs (direct multi-horizon; avoids recursive error accumulation). Loss: MSE on log1p-transformed demand. Optimiser Adam, lr 1e-3, early stopping on validation MAPE.
2. **Prophet** (`ml/prophet_model.py`). Per (node, drug) series, `yearly_seasonality=True`, `weekly_seasonality=True`, Indian holidays via `add_country_holidays('IN')`. Prophet captures trend and seasonality; the LSTM captures short-term nonlinearity — report both, and an ensemble (simple average or inverse-MAPE weighting).
3. **Metrics**. MAPE (primary, per your report), plus RMSE and MASE. **MAPE is undefined when actual demand is zero** — pharmacies have zero-demand days. Use sMAPE or MASE alongside and state this explicitly; an examiner who knows forecasting will ask.
4. Implement the `DemandModel` interface including `get_weights` / `set_weights` — Stage 11 depends on it.
5. Publish daily to `forecast.data`; publish `shortage.alert` when forecast demand exceeds projected stock; publish `demand.hotspot` for the Expiry Agent.

DoD

14-day forecasts for all (node, drug) pairs, with a baseline comparison table: naive / seasonal-naive / Prophet / LSTM / ensemble. Beating seasonal-naive is the bar that matters.

STAGE 8 — Expiry Management Agent ← LLD Box 3

Weeks 5–6 · Owner: A

LLD flow

Batch Data (lot, mfg, exp) → 30-Day Expiry Detector (exp – today ≤ 30) → flagged → Near-Expiry Flag Engine (tag + score) → Auction Negotiation Protocol (bid/match) → matched buyer → Redistribution Generator → **Redistribution Order**

Steps

1. **Expiry Detector** — daily scan; flag batches with `expiry_date - sim_day ≤ 30` (FR-04).
2. **Flag Engine scoring** — urgency score combining days-to-expiry, quantity at risk, unit value, and local forecast demand. Rank descending; this decides auction order.
3. **Auction Protocol** — implement a **sealed-bid second-price (Vickrey) auction**, and say so by name:
4. Seller node announces (drug, quantity, days-to-expiry).
5. Each candidate buyer node computes a private bid = expected value of consuming that quantity before expiry, from its own Demand-Agent forecast, minus transport cost from the Route Agent.
6. Highest bidder wins, pays second-highest price. Vickrey is incentive-compatible (truthful bidding is dominant), which is a defensible, citable design choice rather than an arbitrary heuristic.
7. Reserve price = disposal cost avoided. No bid above reserve ⇒ no redistribution.
8. **FEFO** — First-Expiry-First-Out issuing policy at every node (from your agent-architecture diagram), overriding FIFO.
9. Publish to `redistribution.request`; on execution write a `REDISTRIBUTION` ledger event.

DoD

Wastage (expired units) falls measurably versus a no-redistribution baseline. This is the number behind your abstract's "30–40% wastage reduction" claim — **report what you actually measure**.

STAGE 9 — Route Optimisation Agent ← LLD Box 4

Weeks 5–7 · Owner: B

LLD flow

Delivery Requests → Delivery Node Mapper (geo + priority) → OR-Tools CVRP Solver (capacitated VRP) ← Cold-Chain Temp Constraints (2–8 °C) → Real-time Rerouting Engine ← IoT Alert Trigger (sensor breach?) → **Updated Route Plan**

Steps

1. **Distance matrix** — haversine distance between node lat/lon, scaled to integers (OR-Tools requires integer costs).
2. **CVRP solver** (`agents/routing.py`) using `ortools.constraint_solver.routing_enums_pb2` :
3. `RoutingIndexManager(num_locations, num_vehicles, depot)`
4. Capacity dimension per vehicle.
5. Time-window dimension for delivery windows (FR-05).
6. First solution: `PATH_CHEAPEST_ARC` ; metaheuristic: `GUIDED_LOCAL_SEARCH` ; time limit 10 s.
7. **Cold-chain constraint** — cold-chain batches may only be assigned to refrigerated vehicles. Model as a **disjunction/allowed-vehicle constraint**, not a post-hoc filter — post-filtering silently produces infeasible plans.
8. **Genetic Algorithm** (shown in your agent-architecture diagram) — implement a simple GA as a *comparison baseline* against OR-Tools. Reporting "OR-Tools converges to within X% of optimum in 1/10th the time of our GA" is a legitimate experimental result and justifies the diagram.
9. **Real-time rerouting** — subscribe to cold-chain excursion events; on breach, re-solve for affected shipments only (warm-start from the current solution).
10. Validate against **CVRPLIB** instances with known optima — report your gap percentage.

DoD

Feasible routes respecting capacity, time windows, and cold chain; excursion triggers a re-solve; CVRPLIB gap reported.

STAGE 10 — Anomaly Detection Agent ← LLD Box 5

Weeks 6–7 · Owner: C

LLD flow (with the ledger substitution applied)

`Shipment Log Input` → `Isolation Forest` (outlier scoring) || `Autoencoder` (reconstruction error) → `Anomaly Check` (score > τ) → suspect → `Ledger Hash Verify` (SHA-256 chain cross-check) → mismatch → **Counterfeit / Fraud Alert**

Steps

1. **Features** from shipment logs: transit-time deviation from expected, quantity deviation, temperature excursion count/duration, unusual route, delivery-time-of-day, node-pair frequency.
2. **Isolation Forest** (`sklearn.ensemble.IsolationForest` , `contamination=0.05` , 200 estimators) → outlier score.
3. **Autoencoder** (PyTorch): encoder 12→8→4, decoder 4→8→12, trained only on normal shipments; reconstruction error is the anomaly score. Threshold τ = 95th percentile of training reconstruction error.
4. **Ensemble** — flag if either model exceeds threshold (high recall), or both (high precision). Report the precision/recall trade-off for each rule; your report's reward function for this agent is precision + recall, so you need both numbers.
5. **Ledger Hash Verify** — for each suspect shipment, call `verify_batch_fingerprint()` and `get_provenance()` . A fingerprint mismatch, a missing expected handoff, or a broken chain segment ⇒ **Counterfeit / Fraud Alert**. This is the concrete realisation of research gap 3.2.3: the ledger is a live input to an AI decision system, not an isolated record store.
6. **Labels** from openFDA recall data plus synthetically injected anomalies (inject known counterfeits into the simulation so you have ground truth). Report precision, recall, F1, ROC-AUC.

DoD

Detects injected counterfeit batches; confusion matrix and ROC curve produced; ledger cross-check demonstrably catches a forged fingerprint that the ML models alone rank as borderline.

★ STAGE 10.5 — INTEGRATION GATE

End of Week 7 · Owner: all three · **THE MOST IMPORTANT MILESTONE**

Do not proceed until all of the following hold simultaneously in a single run:

- [] Simulation runs 365 days over ≥ 12 nodes without error
- [] All five agents observe, decide, act every simulated day
- [] Every physical handoff produces a signed, chained ledger record
- [] `verify_chain()` returns `True` at end of run; tampering makes it `False`
- [] KPIs computed and printed: stockout rate, wastage units, forecast MAPE, delivery cost, anomaly precision/recall
- [] Every agent decision is persisted in `agent_decisions`
- [] `pytest` passes; coverage $\geq 60\%$ (drive to 80% by Stage 15)
- [] A single command (`make sim`) reproduces everything from a clean checkout

At this point you have a complete, effective, demonstrable capstone project. Tag this commit `v1.0-integrated` . Everything after this stage is research depth, and every subsequent stage is individually optional and independently valuable.

STAGE 11 — Federated Learning ← Layer 5 of your HLD

Weeks 7–9 · Owner: C

Goal

Convert the centralised demand model into a federated one, delivering FR-08, NFR-03, and closing research gap 3.2.4.

Where

`pharmadt/federated/client.py`, `server.py`, `partition.py`, `privacy.py`

Steps

1. **Non-IID partition** (`partition.py`) — split Rossmann/CMS data across simulated hospital/pharmacy clients using a **Dirichlet distribution** with concentration $\alpha = 0.5$. This produces realistic label/volume skew. Also produce an IID split for comparison; the IID-vs-non-IID gap is one of your best experimental results.
2. **Flower client** (`client.py`) — subclass `flwr.client.NumPyClient`, implementing `get_parameters`, `fit`, `evaluate`, delegating to your `DemandModel.get_weights / set_weights` from Stage 1.
3. **Server** (`server.py`) — `flwr.server.strategy.FedAvg(fraction_fit=1.0, min_fit_clients=N)`. Rounds: 30–50. Local epochs E: 1, 3, 5 (this is an ablation axis).
4. **Differential privacy** (`privacy.py`) — clip each client's gradient to L2 norm C, add Gaussian noise $\mathcal{N}(0, \sigma^2 C^2)$. Track the privacy budget (ϵ, δ) with a moments accountant (`opacus.accountants.RDPAccountant`). Report accuracy at $\epsilon \in \{1, 5, 10, \infty\}$. **The accuracy-vs-privacy curve is one of the strongest figures you can put in the final report.**
5. **Fallback**: if non-IID divergence degrades accuracy badly, switch `FedAvg` → `FedProx` (proximal term $\mu = 0.01$). Your report already anticipates this (§3.1.4 weaknesses), so demonstrating the fix is a direct contribution.
6. Plug the aggregated global model back into the Demand Agent and re-run the full simulation.

DoD

$N \geq 5$ clients train for ≥ 30 rounds; comparison table of centralised vs federated-IID vs federated-non-IID vs federated+DP, reported in MAPE. **Raw demand data never leaves a client** — assert this in a test to evidence NFR-03.

STAGE 12 — MARL Upgrade (MADDPG)

Weeks 9–12 · Owner: B · highest risk, highest research value

Goal

Replace heuristic decision-making in the coordination-heavy agents with learned policies, closing research gap 3.2.2.

Steps

1. **Environment wrapper** (`marl/env.py`) — wrap the SimPy twin as a PettingZoo `ParallelEnv` with `reset()`, `step(actions)`, `observation_space`, `action_space`. This is straightforward *only because* Stage 3 kept agents decoupled from SimPy internals.
2. **Spaces** (`marl/spaces.py`):
3. **Observation** (per agent): the node state vector from Stage 3 + relevant global signals. Normalise everything to roughly $[-1, 1]$ — unnormalised observations are the number-one cause of MADDPG failing to learn.
4. **Action**: Inventory → continuous order quantity per drug; Route → continuous priority weights fed into OR-Tools; Expiry → continuous redistribution aggressiveness.
5. **Rewards** (`marl/rewards.py`) — exactly as your report §5.2.2 specifies: Inventory = fill rate – holding cost; Expiry = value of prevented wastage; Route = negative delivery cost; Demand = –MAPE; Anomaly = precision + recall. Scale each to comparable magnitude or one agent's reward will dominate.
6. **MADDPG** (`marl/maddpg.py`) — centralised training, decentralised execution. Per agent: actor $\pi(o)$ and critic $Q(x, a_1 \dots a_n)$ seeing all observations and actions. Replay buffer 10%, soft target updates $\tau = 0.01$, $\gamma = 0.95$, Ornstein-Uhlenbeck or Gaussian exploration noise decaying over training.
7. **Start with two agents** (Inventory + Route), then add Expiry. Five simultaneously learning agents rarely converge on a first attempt.
8. **Benchmark** against Stage 6–9 heuristics and a classical OR baseline.

DoD

Training curves showing reward improvement; benchmark table (heuristic vs classical OR vs MADDPG). **If MADDPG underperforms, report that honestly** — a negative result with correct methodology and a clear analysis of *why* (non-stationarity, reward scaling, sample complexity) is a legitimate and respected research finding. Your heuristic agents remain the shipping system either way.

Pitfalls

Budget the full three weeks. Log every hyperparameter with each run (use `experiments/` configs). Do not delete the heuristic agents.

STAGE 13 — Crisis Simulation Engine

Weeks 11–13 · Owner: A

Goal

FR-09 — parameterised what-if scenarios.

Steps

1. **Scenario definitions** in YAML (`data/scenarios/`), each specifying trigger day, duration, magnitude, affected nodes:
2. `pandemic_surge.yaml` — demand $\times 10$ for 60 days across all pharmacies
3. `factory_shutdown.yaml` — manufacturer capacity $\rightarrow 0$ for 30 days
4. `coldchain_failure.yaml` — refrigeration failure on X% of shipments during a heatwave window
5. `route_disruption.yaml` — edges removed from the graph (flood/landslide)
6. **Injector** (`crisis/injector.py`) — a SimPy process that modifies world state at the trigger day, cleanly reversible so scenarios compose.
7. **Metrics** — time-to-detect, time-to-recover (days until stockout rate returns to baseline), peak stockout, total unmet demand, cost of response.
8. **Run each scenario under both heuristic and MADDPG policies.** The comparison is the payoff for Stage 12.

DoD

Four scenarios run end-to-end; a resilience comparison table and recovery-curve plots produced.

STAGE 14 — API & Dashboard ← *Layer 6 of your HLD*

Weeks 12–14 · Owner: C (+A on API)

Backend (FastAPI)

Routes: `POST /simulation/run` , `GET /simulation/state` , `GET /agents/decisions` , `GET /ledger/provenance/{batch_id}` , `GET /ledger/verify` , `GET /kpi` , `POST /crisis/inject` . `WebSocket /ws/events` streams live simulation events. OAuth2 password flow satisfies NFR-04's API requirement (`fastapi.security.OAuth2PasswordBearer`).

Frontend (React + Vite)

Components mapping one-to-one to your HLD dashboard panel:

HLD element	Component	Library
Network Status Map	<code>NetworkMap</code>	react-leaflet + node markers coloured by stock health
Agent Decision Logs	<code>AgentLog</code>	virtualised table, live via WebSocket
What-If Scenarios	<code>CrisisControls</code>	scenario picker + run button
Verification Panel	<code>ProvenanceTrace</code>	batch lookup $\rightarrow$ chain trace + green/red chain-integrity badge
Wastage / Stockout / Compliance	<code>KPICards</code> + <code>KPICharts</code>	Recharts

Highest-impact demo element: the Verification Panel showing a live chain-integrity badge that flips red when you tamper with a record. It makes the cryptographic ledger visible and immediately convincing.

DoD

Dashboard loads in under 3 seconds (NFR-06), displays live simulation state, and a non-technical viewer can run a crisis scenario unaided.

STAGE 15 — Evaluation, Ablation & Packaging

Weeks 14–16 · Owner: all three

Experiment matrix (this table is the core of your final report)

Configuration	Stockout %	Wastage units	MAPE	Delivery cost	Anomaly F1
Baseline (no agents, fixed reorder)					
+ Inventory & Demand agents					
+ Expiry agent (redistribution)					
+ Route optimisation					
+ Anomaly detection & ledger					
+ Federated (non-IID)					
+ Federated + DP ($\epsilon=5$)					
+ MADDPG					
Full system under each crisis scenario					

Run every configuration with ≥ 10 random seeds and report **mean $\pm$ standard deviation**. Single-run numbers are not defensible.

Validate your abstract’s claims

Your abstract promises 30–40% wastage reduction and 20–25% forecast improvement. Compute both from the ablation table. If measured values differ, **update the abstract to the measured values** — a defended real number beats an undefended target every time.

Packaging

- `docker compose up` brings up database, API, and dashboard from a clean checkout.
- README with architecture diagram, setup, and a reproduction script.
- `pytest --cov` $\geq 80\%$ (NFR-07).
- Tag `v2.0-final` ; archive with a DOI via Zenodo if your department permits.

PART E — Risk Register & Contingency

Risk	Impact	Likelihood	Mitigation / trigger
MADDPG does not converge	High	High	Heuristics from Stages 6–10 ship the project. Trigger: if no reward improvement after 2 weeks of Stage 12, freeze MADDPG, report the negative result with analysis, move to Stage 13
FL diverges on non-IID data	Medium	Medium	Switch FedAvg → FedProx ($\mu=0.01$); reduce local epochs; report the comparison as a finding
Digital twin slips schedule	Critical	Medium	It is the critical path. If Stage 3 is not done by end of Week 4, cut node count to 8 and defer the crisis engine
Prophet/PyTorch install failures	Medium	Medium	Use WSL2 or Docker for the whole dev environment; pin every version
Ledger appears "too simple" to an examiner	Low	Low	Merkle anchoring + ECDSA + DB-enforced immutability + the tamper demo. Use the Part A justification paragraph verbatim
Integration failures late in the project	High	Low	The Stage 10.5 gate exists precisely to eliminate this
Scope creep	High	High	Stages 11–13 are individually optional. Cut from the <i>end</i> , never from the middle

PART F — Weekly Checkpoint Schedule

Week	Milestone	Evidence to show your guide
1	Stage 0–1	Repo boots, schema migrates, tests pass
2	Stage 2 + Stage 3 start	Processed datasets; network graph renders
3	Stage 3 mid + Stage 4 start	Simulation runs 100 days; ledger appends
4	Stage 3 + 4 done; Stage 5 done	Tamper demo ; agents plug into bus
5	Stages 6, 8	Inventory + Expiry acting on the twin
6	Stages 7, 10	Forecasts with MAPE; anomaly confusion matrix
7	Stage 9 + ★ GATE	Full autonomous loop; tag v1.0-integrated
8–9	Stage 11	FL comparison table; privacy curve
10–12	Stage 12	Training curves; MARL vs heuristic benchmark
11–13	Stage 13	Four crisis scenarios; recovery curves
12–14	Stage 14	Live dashboard demo
14–16	Stage 15	Full ablation matrix; final report; v2.0-final

PART G — Ownership Summary

Owner	Primary stages	Core deliverables
A (Twin & Simulation)	1, 3, 8, 13	Digital twin engine, domain model, expiry/auction agent, crisis engine
B (Agents & MARL)	5, 6, 9, 12	Agent framework & bus, inventory agent, route optimisation, MADDPG
C (Ledger, ML, FL, UI)	2, 4, 7, 10, 11, 14	Cryptographic ledger, datasets, demand forecasting, anomaly detection, federated learning, dashboard

All three co-own Stage 0, the Stage 10.5 gate, and Stage 15.

PART H — Keeping It "Complex Yet Legible"

Your stated design goal — opaque to a layperson, transparent to an ML engineer — is achieved by these specific practices, not by adding complexity:

1. **Every component behind a documented ABC.** A reader traces `ProvenanceLedger` or `Agent` and immediately understands the contract without reading implementations.
2. **Each module under ~300 readable lines.** Depth comes from composition, not from long functions.
3. **Data contracts documented per agent** — observation space, action space, reward — as a short table in the README.
4. **Named, citable algorithms throughout.** Vickrey auction, FedAvg/FedProx, MADDPG, Isolation Forest, Guided Local Search, ECDSA P-256, Merkle inclusion proof. Named methods signal rigour and are individually defensible in a viva; ad-hoc heuristics are not.
5. **Cryptographic depth in place of infrastructure depth.** Hash chaining, elliptic-curve signatures, and Merkle proofs read as serious to any examiner while remaining a few hundred lines an ML engineer reads in one sitting — precisely the balance you asked for.
6. **Every claim traceable to a measurement.** The ablation matrix means any number in your report can be regenerated by one command.